

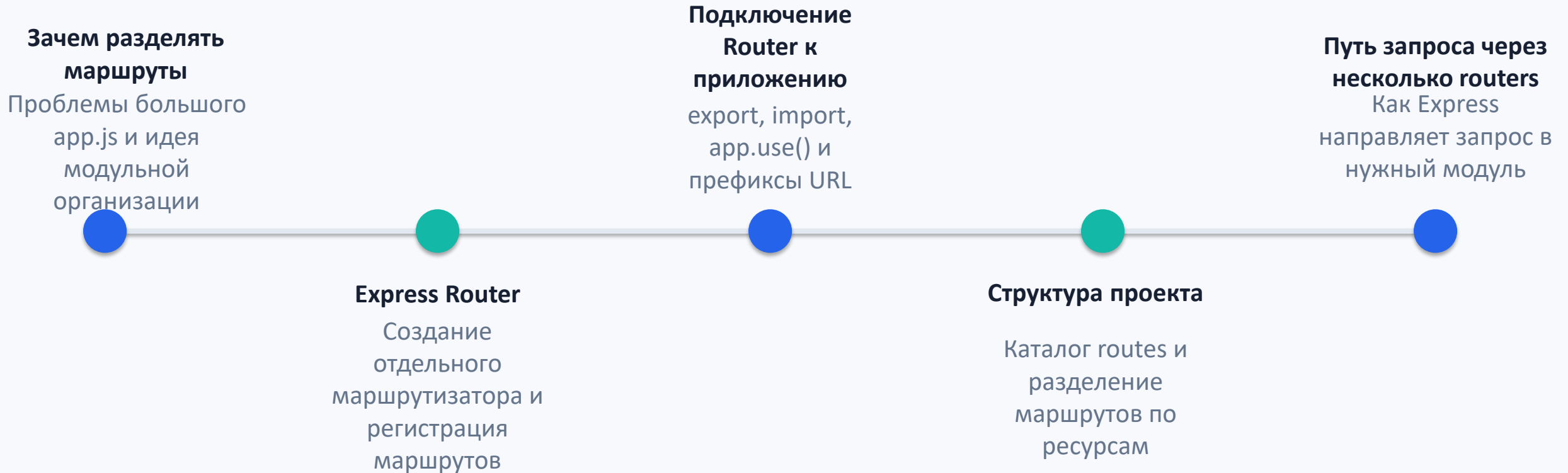
Express Router и организация структуры проекта

Разделяем маршруты серверного приложения на независимые модули

Результаты занятия

- ✓ Объяснять, почему маршруты большого приложения необходимо разделять по модулям
- ✓ Создавать маршрутизатор с помощью `express.Router()`
- ✓ Регистрировать маршруты через `router.get()`, `router.post()`, `router.patch()` и другие методы
- ✓ Экспортировать и импортировать Router как ES Module
- ✓ Подключать Router к приложению через `app.use()`
- ✓ Понимать формирование полного URL из префикса `app.use()` и пути Router
- ✓ Организовывать каталог routes для нескольких ресурсов приложения

Учебные вопросы





Учебный вопрос 1. Зачем разделять маршруты

Почему один app.js перестаёт быть удобным по мере роста приложения

Начать можно с одного файла

- В небольшом учебном приложении все маршруты удобно разместить непосредственно в `app.js`.
- В одном месте находятся создание `app`, подключение `middleware`, маршруты и запуск сервера.
- Такую структуру легко понять на первых занятиях.
- Для нескольких `endpoints` дополнительное разделение действительно не требуется.

Главная идея

Проблема возникает не сразу, а по мере увеличения количества ресурсов и маршрутов.

Когда app.js начинает расти

```
app.get("/api/products", ...);  
app.get("/api/products/:id", ...);  
app.post("/api/products", ...);  
app.patch("/api/products/:id", ...);  
app.delete("/api/products/:id", ...);
```

```
app.get("/api/categories", ...);  
app.post("/api/categories", ...);
```

```
app.get("/api/orders", ...);  
app.post("/api/orders", ...);  
app.patch("/api/orders/:id", ...);
```

Проблемы большого app.js

● Много кода

Десятки маршрутов разных ресурсов находятся в одном файле.

● Сложнее искать

Для изменения товара приходится искать нужный обработчик среди несвязанных маршрутов.

● Сложнее изменять

Любое расширение увеличивает главный файл приложения.

● Сложнее сопровождать

Структура приложения становится менее очевидной для разработчика.

От большого app.js к модульной маршрутизации

Тот же Express.
Просто лучше организован!

```
app.js JS
1  const express = require('express')
2  const app = express()
3
4  // ... middleware, настройки и т.д.
5
6  // Продукты
7  app.get("/api/products", ...)
8  app.post("/api/products", ...)
9
10 // Категории
11 app.get("/api/categories", ...)
12 app.post("/api/categories", ...)
13
14 // Заказы
15 app.get("/api/orders", ...)
16 app.post("/api/orders", ...)
17
18 // Снова продукты (где-то ниже...)
19 app.get("/api/products/:id", ...)
20
21 // Снова заказы (ещё ниже...)
22 app.patch("/api/orders/:id", ...)
23
24 // ... и ещё много маршрутов вперемешку
```

Все маршруты в одном файле

Разделение
по ресурсам



products



categories



orders

```
app.js JS
1  const express = require('express')
2  const app = express()
3
4  const productRoutes = require('./routes/productRoutes')
5  const categoryRoutes = require('./routes/categoryRoutes')
6  const orderRoutes = require('./routes/orderRoutes')
7
8  app.use("/api/products", productRoutes)
9  app.use("/api/categories", categoryRoutes)
10 app.use("/api/orders", orderRoutes)
11
12 // ... остальные настройки и запуск сервера
```

```
productRoutes.js JS
1  const express = require('express')
2  const router = express.Router()
3
4  router.get("/", ...)
5  router.get("/:id", ...)
6  router.post("/", ...)
7
8  module.exports = router
```

Маршруты для продуктов

```
categoryRoutes.js JS
1  const express = require('express')
2  const router = express.Router()
3
4  router.get("/", ...)
5  router.post("/", ...)
6
7
8  module.exports = router
```

Маршруты для категорий

```
orderRoutes.js JS
1  const express = require('express')
2  const router = express.Router()
3
4  router.get("/", ...)
5  router.post("/", ...)
6  router.patch("/:id", ...)
7
8  module.exports = router
```

Маршруты для заказов

Маршруты организованы по ресурсам



Функциональность не меняется — структура проекта становится понятнее и удобнее в сопровождении.



Группируем код по ресурсам

- Все маршруты товаров относятся к одному ресурсу `products`.
- Маршруты категорий относятся к ресурсу `categories`.
- Заказы образуют отдельную группу `routes` для `orders`.
- Каждую такую группу можно хранить в собственном JavaScript-файле.
- Главный `app.js` будет только подключать готовые маршрутизаторы.

Будущие модули маршрутизации

Ресурс	Файл	Примеры endpoints
Products	productRoutes.js	GET /, GET /:id, POST /, PATCH /:id
Categories	categoryRoutes.js	GET /, POST /
Orders	orderRoutes.js	GET /, POST /, PATCH /:id
Application	app.js	Подключение готовых routers



Учебный вопрос 2. Express Router

Создаём отдельную систему маршрутизации для одного ресурса

Что такое Express Router

- `express.Router()` создаёт отдельный маршрутизатор, в котором можно регистрировать маршруты и `middleware` независимо от основного объекта `app`.
- Router поддерживает привычные HTTP-методы: `get()`, `post()`, `put()`, `patch()`, `delete()`.
- Он может содержать маршруты только одного логического раздела приложения.
- Router можно экспортировать из файла и подключить в другом модуле.
- Один Express-проект может одновременно использовать несколько Router.

Главная идея

Express Router часто называют мини-приложением, потому что он представляет отдельную систему маршрутизации и `middleware`.

app и router

app

- Главный объект Express-приложения
- Создаётся через `express()`
- Подключает глобальные `middleware`
- Подключает `routers`
- Запускает HTTP-сервер

router

- Отдельный маршрутизатор
- Создаётся через `express.Router()`
- Хранит группу связанных `endpoints`
- Может использовать `middleware`
- Монтируется внутрь `app`

Express Router как мини-приложение

Один проект
Много маршрутов
Больше возможностей!

Разные роутеры —
одна цель:
вместе в одном
Express-приложении



Express-приложение app

```
const express = require('express')  
const app = express()
```

Роутеры подключены внутри приложения
(через `app.use(...)`)



Products Router

/products

GET	/	— список товаров
GET	/:id	— товар по ID
POST	/	— создать товар
PATCH	/:id	— обновить товар
DELETE	/:id	— удалить товар

```
app.use('/products', productsRouter)
```



Categories Router

/categories

GET	/	— список категорий
POST	/	— создать категорию
PATCH	/:id	— обновить категорию
DELETE	/:id	— удалить категорию

```
app.use('/categories', categoriesRouter)
```



Orders Router

/orders

GET	/	— список заказов
GET	/:id	— заказ по ID
POST	/	— создать заказ
PATCH	/:id	— обновить заказ
DELETE	/:id	— удалить заказ

```
app.use('/orders', ordersRouter)
```



Routers — часть одного Express-приложения и работают через один HTTP-сервер.



Создаём новый Router

```
import express from "express";  
const router = express.Router();
```

Что важно

- Express импортируется как обычная npm-зависимость.
- `express.Router()` создаёт объект маршрутизатора.
- После этого маршруты регистрируются через `router` вместо `app`.
- Создание `Router` не запускает отдельный HTTP-сервер.

Первый маршрут внутри Router

```
router.get("/", (req, res) => {  
  res.json(products);  
});
```

Что важно

- `router.get()` работает аналогично `app.get()`.
- Путь `"/"` является локальным путём данного Router.
- Полный URL пока не указан.
- Он будет сформирован после подключения Router в `app.js`.

Динамический маршрут внутри Router

```
router.get("/:id", (req, res) => {  
  const id = Number(req.params.id);  
  const product = products.find(p => p.id === id);  
  
  res.json(product);  
});
```

Что важно

- Параметры маршрутов работают внутри Router так же, как внутри app.
- Путь `/:id` будет добавлен к префиксу Router.
- `req.params.id` содержит динамическое значение из URL.
- Обработчик может использовать `req`, `res` и все знакомые возможности Express.

Группа маршрутов products

```
router.get("/", getProducts);  
router.get("/:id", getProductById);  
router.post("/", createProduct);  
router.patch("/:id", updateProduct);  
router.delete("/:id", deleteProduct);
```

Почему пути стали короче

- В `productRoutes.js` все маршруты относятся к одному ресурсу.
- Поэтому нет необходимости повторять `/api/products` в каждой строке.
- Общий префикс будет указан один раз при подключении Router.
- Внутри Router остаются только части пути, различающие endpoints.

Главная идея

Это уменьшает повторение и делает группу маршрутов легче для чтения.

Учебный вопрос 3. Подключение Router к приложению

Экспортируем маршрутизатор и монтируем его через `app.use()`

Экспортируем Router

```
import express from "express";  
  
const router = express.Router();  
  
router.get("/", (req, res) => {  
  res.json(products);  
});  
  
export default router;
```

Что важно

- Router является обычным JavaScript-объектом.
- Его можно экспортировать из модуля.
- `export default` позволяет импортировать Router в `app.js`.
- Файл маршрутов сам сервер не запускает.

Импортируем Router в app.js

```
import express from "express";  
import productRoutes from "../routes/productRoutes.js";  
  
const app = express();
```

Что важно

- app.js импортирует готовый маршрутизатор.
- Относительный путь указывает на файл внутри каталога routes.
- Расширение .js указывается явно при использовании ES Modules.
- Теперь productRoutes можно подключить к приложению.

Монтируем Router через app.use()

```
app.use("/api/products", productRoutes);
```

Что важно

- Первый аргумент задаёт общий префикс маршрутов.
- Второй аргумент — подключаемый Router.
- Все маршруты `productRoutes` теперь начинаются с `/api/products`.
- Express направляет подходящие запросы внутрь этого Router.

Один Express-проект — много маршрутов с понятной структурой!

Как формируется полный URL Router

Маленькие Router — большие возможности!



Как объединяются пути

Полный URL получается из префикса, указанного в `app.use()`, и пути, заданного внутри Router.

Префикс `app.use()`

(mount path)

```
app.use("/api/products", productRoutes)
```

Подключаем Router по этому пути

+

Локальный путь Router

(route path)

```
router.get("/:id")
```

Путь внутри Router (относительно префикса)

→

Итоговый endpoint

(полный URL)

```
GET /api/products/:id
```

Из `app.use()` (mount path)

Из Router (route path)



Пример 1: получаем список товаров

```
app.use("/api/products", productRoutes)
```

+

```
router.get("/")
```

→

```
GET /api/products
```

Префикс `app.use()` (mount path)

Локальный путь Router

Итоговый endpoint



Пример 2: удаляем товар по ID

```
app.use("/api/products", productRoutes)
```

+

```
router.delete("/:id")
```

→

```
DELETE /api/products/:id
```

Префикс `app.use()` (mount path)

Локальный путь Router

Итоговый endpoint



Полный URL = префикс подключения + путь внутри Router

Простые правила.
Рабочие маршруты.
Больше возможностей!



Префикс `app.use()` + путь `router`

Префикс <code>app.use()</code>	Путь Router	Итоговый endpoint
<code>/api/products</code>	<code>/</code>	GET <code>/api/products</code>
<code>/api/products</code>	<code>/:id</code>	GET <code>/api/products/:id</code>
<code>/api/products</code>	<code>/</code>	POST <code>/api/products</code>
<code>/api/products</code>	<code>/:id</code>	DELETE <code>/api/products/:id</code>

Запрос конкретного товара

Запрос: GET /api/products/15

- 1 Express видит префикс /api/products
- 2 Запрос передаётся в productRoutes
- 3 Router сопоставляет оставшийся путь /15 с /:id
- 4 Значение 15 становится req.params.id
- 5 Запускается соответствующий обработчик

Путь запроса через app.use() и Router

Как Express определяет нужный Router и передаёт запрос в соответствующий обработчик

1 Клиент

Отправляет HTTP-запрос



HTTP-клиент

Браузер, Postman, мобильное приложение и т.д.

GET /api/products/15

HTTP/1.1

Host: localhost:3000

Accept: application/json

Запрос

GET
/api/products/15

2 Express app

Проверяет маршруты и префиксы



Express Application
app.js

```
import express from "express";
import productRoutes from "../routes/productRoutes.js";

const app = express();
app.use(express.json());

app.use("/api/products", productRoutes);
app.use("/api/categories", categoryRoutes);
app.use("/api/orders", orderRoutes);

app.listen(3000, () => {
  console.log("Server started");
});
```

Передаёт
в Router

Оставшаяся часть
пути: /15

Express находит совпадение

Префикс /api/products
совпадает с app.use("/api/products", ...)
Запрос передаётся в productRoutes.

3 Product Router

Ищет нужный маршрут



routes/productRoutes.js

```
import express from "express";

const router = express.Router();

router.get("/", getProducts);
router.get("/:id", getProductById);
router.post("/", createProduct);
router.patch("/:id", updateProduct);
router.delete("/:id", deleteProduct);

export default router;
```

Выполняет
обработчик

Совпадает маршрут /:id

Внутри Router путь /15
соответствует маршруту /:id.
Вызывается обработчик
getProductById.

4 Обработчик

Обрабатывает запрос

```
const getProductById = (req, res) => {
  const id = Number(req.params.id);

  const product = products.find(p =>
    p.id === id);

  if (!product) {
    return res.status(404).json({
      message: "Product not found"
    });
  }

  res.json(product);
};
```

req.params.id = "15"

Из URL параметр 15 попадает
в req.params.id.
Обработчик находит товар
и отправляет JSON-ответ.

5 Ответ клиенту

JSON-данные возвращаются
обратно

HTTP/1.1 200 OK

Content-Type: application/json

```
{
  "id": 15,
  "name": "Gaming Laptop",
  "price": 120000,
  "inStock": true
}
```

JSON-ответ

Товар с id = 15



Все Router работают внутри
одного Express-приложения
и используют один HTTP-сервер.

app.use() здесь работает как точка подключения

- app.use() связывает определённый URL-префикс с Router.
- Запросы к другим префиксам могут направляться в другие routers.
- Это позволяет главному приложению выполнять роль диспетчера.
- Внутреннее устройство Router при этом скрыто от app.js.

Главная идея

app.js знает, какой Router отвечает за ресурс, но ему не нужно содержать сами маршруты.

Минимальный app.js после разделения

```
import express from "express";
import productRoutes from "../routes/productRoutes.js";

const app = express();
const PORT = 3000;

app.use(express.json());
app.use("/api/products", productRoutes);

app.listen(PORT, () => {
  console.log(`Server: http://localhost:${PORT}`);
});
```

До и после Express Router

До

- app.js содержит все маршруты
- Постоянно повторяется /api/products
- Файл быстро увеличивается
- Ресурсы смешаны между собой

После

- app.js подключает готовые Router
- Префикс задаётся один раз
- Маршруты разделены по ресурсам
- Структура приложения видна по каталогам



Учебный вопрос 4. Организация структуры проекта

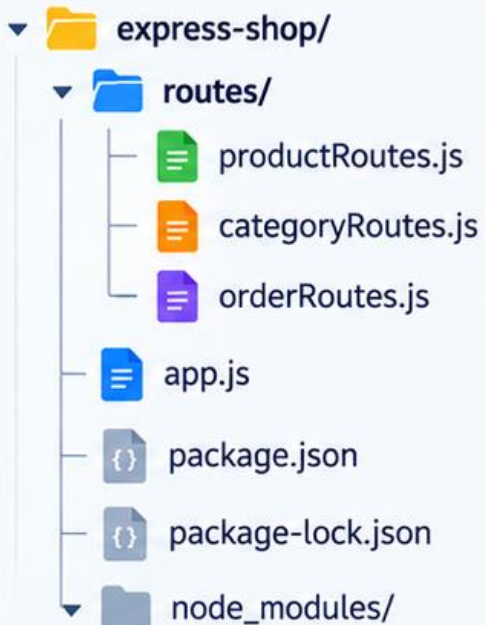
Создаём каталог routes и разделяем разные ресурсы

Структура Express-проекта с несколькими routers



Разделяем маршруты по ресурсам и подключаем их в основном файле приложения

Структура проекта express-shop/



Файл app.js подключает маршруты из каталога routes и монтирует их на соответствующие URL-префиксы.

```
app.js (главный файл приложения)

1 import express from "express";
2 import productRoutes from "../routes/productRoutes.js";
3 import categoryRoutes from "../routes/categoryRoutes.js";
4 import orderRoutes from "../routes/orderRoutes.js";
5
6 const app = express();
7 const PORT = 3000;
8
9 app.use(express.json());
10
11 // Подключаем routers с разными префиксами
12 app.use("/api/products", productRoutes);
13 app.use("/api/categories", categoryRoutes);
14 app.use("/api/orders", orderRoutes);
15
16 app.listen(PORT, () => {
17   console.log(`Server started: http://localhost:${PORT}`);
18 });
```



Основной файл остаётся компактным:

только настройки, подключение routers и запуск сервера.

routes/productRoutes.js

Маршруты товаров (Products)

```
1 import express from "express";
2 const router = express.Router();
3
4 router.get("/", getProducts);
5 router.get("/:id", getProductById);
6 router.post("/", createProduct);
7 router.patch("/:id", updateProduct);
8 router.delete("/:id", deleteProduct);
9 export default router;
```

Endpoints

GET /
GET /:id
POST /
PATCH /:id
DELETE /:id

routes/categoryRoutes.js

Маршруты категорий (Categories)

```
1 import express from "express";
2 const router = express.Router();
3
4 router.get("/", getCategories);
5 router.post("/", createCategory);
6 router.get("/:id", getCategoryById);
7 router.patch("/:id", updateCategory);
8 export default router;
```

Endpoints

GET /
POST /
GET /:id
PATCH /:id

routes/orderRoutes.js

Маршруты заказов (Orders)

```
1 import express from "express";
2 const router = express.Router();
3
4 router.get("/", getOrders);
5 router.get("/:id", getOrderById);
6 router.post("/", createOrder);
7 router.patch("/:id", updateOrder);
8 export default router;
```

Endpoints

GET /
GET /:id
POST /
PATCH /:id



Преимущества:



Чёткая структура проекта



Лёгкое добавление новых ресурсов



Удобное сопровождение



Возможность командной разработки

Каталог routes

- Для файлов маршрутизации удобно создать отдельный каталог routes.
- Каждый ресурс получает свой файл Router.
- Имя файла должно ясно показывать, какой ресурс он обслуживает.
- Главный app.js остаётся в корне проекта или внутри каталога src.
- Такое разделение создаёт основу для дальнейшего развития архитектуры.

Простая структура проекта

```
express-shop/  
├── routes/  
│   ├── productRoutes.js  
│   ├── categoryRoutes.js  
│   └── orderRoutes.js  
├── app.js  
├── package.json  
└── package-lock.json
```

Ответственность файлов

Файл	Ответственность
app.js	Создание приложения, общие middleware, подключение routers, запуск сервера
productRoutes.js	Маршруты ресурса products
categoryRoutes.js	Маршруты ресурса categories
orderRoutes.js	Маршруты ресурса orders

categoryRoutes.js

JAVASCRIPT

```
import express from "express";

const router = express.Router();

router.get("/", (req, res) => {
  res.json(categories);
});

router.post("/", (req, res) => {
  res.status(201).json(req.body);
});

export default router;
```

Что важно

- Этот Router работает только с категориями.
- Его локальные пути начинаются с "/".
- Общий префикс будет задан в app.js.
- Файл экспортирует только созданный Router.

orderRoutes.js

```
import express from "express";

const router = express.Router();

router.get("/", (req, res) => {
  res.json(orders);
});

router.post("/", (req, res) => {
  res.status(201).json(req.body);
});

export default router;
```

Что важно

- Структура Router повторяется для другого ресурса.
- Каждый модуль можно изменять независимо.
- URL-префикс снова не дублируется внутри файла.
- Подключение выполняется централизованно в app.js.

Подключаем несколько Router

```
import productRoutes from "../routes/productRoutes.js";
import categoryRoutes from "../routes/categoryRoutes.js";
import orderRoutes from "../routes/orderRoutes.js";

app.use("/api/products", productRoutes);
app.use("/api/categories", categoryRoutes);
app.use("/api/orders", orderRoutes);
```

Карта маршрутизаторов приложения

Префикс	Router	Ресурс
/api/products	productRoutes	Товары
/api/categories	categoryRoutes	Категории
/api/orders	orderRoutes	Заказы

Не нужно создавать отдельное Express-приложение

- Для каждого файла routes не вызывается `express()` повторно.
- Главный объект `app` остаётся один.
- В модулях создаются только экземпляры `express.Router()`.
- Все routers работают внутри одного HTTP-сервера.
- Один порт обслуживает всё приложение.

Главная идея

Router — это часть приложения, а не отдельный сервер.



Учебный вопрос 5. Путь запроса через несколько routers

Как главный app направляет запрос в нужный модуль

Express-приложение как диспетчер маршрутизаторов

Главное приложение (app) определяет нужный Router по URL-префиксу, а Router выбирает конкретный обработчик

HTTP-клиенты

Отправляют запросы к API

GET
/api/products

GET
/api/categories

POST
/api/orders



app.js

Главное Express-приложение

```
import express from "express";
import productRoutes from "../routes/productRoutes.js";
import categoryRoutes from "../routes/categoryRoutes.js";
import orderRoutes from "../routes/orderRoutes.js";

const app = express();
app.use(express.json());

// Подключаем маршрутизаторы
app.use("/api/products", productRoutes);
app.use("/api/categories", categoryRoutes);
app.use("/api/orders", orderRoutes);

app.listen(3000, () => {
  console.log("Server started on port 3000");
});
```

app.js работает как диспетчер:

- проверяет URL-префикс запроса
- передаёт запрос в соответствующий Router
- все маршруты работают на одном HTTP-сервере



productRoutes.js

Router для товаров

```
import express from "express";
const router = express.Router();

router.get("/", getProducts);
router.get("/:id", getProductById);
router.post("/", createProduct);
router.patch("/:id", updateProduct);
router.delete("/:id", deleteProduct);
export default router;
```

Endpoints (Products)

Method	Endpoint
GET	/
GET	/:id
POST	/
PATCH	/:id
DELETE	/:id

JSON-ответ

```
[
  {
    "id": 1,
    "name": "Laptop"
  },
  ...
]
```



categoryRoutes.js

Router для категорий

```
import express from "express";
const router = express.Router();

router.get("/", getCategories);
router.get("/:id", getCategoryById);
router.post("/", createCategory);
router.patch("/:id", updateCategory);
export default router;
```

Endpoints (Categories)

Method	Endpoint
GET	/
GET	/:id
POST	/
PATCH	/:id

JSON-ответ

```
[
  {
    "id": 10,
    "name": "Phones"
  },
  ...
]
```



orderRoutes.js

Router для заказов

```
import express from "express";
const router = express.Router();

router.get("/", getOrders);
router.get("/:id", getOrderById);
router.post("/", createOrder);
router.patch("/:id", updateOrder);
export default router;
```

Endpoints (Orders)

Method	Endpoint
GET	/
GET	/:id
POST	/
PATCH	/:id

JSON-ответ

```
[
  {
    "id": 100,
    "status": "new"
  },
  ...
]
```



Итого:

app определяет нужный Router по URL-префиксу, а каждый Router самостоятельно обрабатывает свои маршруты. Все routers работают внутри одного Express-приложения и используют один HTTP-сервер.



Такой подход делает приложение модульным, понятным и легко расширяемым.

Пример: GET /api/orders

- 1 Клиент отправляет запрос** На сервер приходит GET /api/orders.
- 2 Express проверяет подключения** Префикс /api/orders совпадает с `app.use("/api/orders", orderRoutes)`.
- 3 Запрос передаётся в orderRoutes** Внутри Router остаётся локальный путь /.
- 4 Router находит GET /** Запускается соответствующий обработчик.
- 5 Клиент получает ответ** Обработчик формирует JSON через `res.json()`.

Пример: DELETE /api/products/7

- 1 Префикс /api/products направляет запрос в productRoutes
- 2 Внутренний путь /7 сопоставляется с /:id
- 3 HTTP-метод DELETE выбирает router.delete()
- 4 req.params.id получает строку "7"
- 5 Запускается обработчик удаления товара

Router сохраняет все знакомые возможности Express

- В маршрутах доступны `req.params`, `req.query` и `req.body`.
- Можно использовать методы `res.status()`, `res.json()` и `res.send()`.
- Router поддерживает все основные HTTP-методы.
- К Router можно подключать `middleware`.
- Поэтому перенос маршрута из `app` в Router не меняет принцип его работы.

Где заканчивается ответственность Router

- Router должен описывать endpoints и связывать их с обработчиками.
- По мере роста проекта большие функции-обработчики также стоит выносить из routes.
- Маршрутизация, обработка запроса и бизнес-логика — разные задачи.
- На следующем этапе маршруты будут связаны с отдельными controllers и services.

Главная идея

Сегодня разделяем маршруты. На следующем занятии продолжим разделять ответственность компонентов приложения.

Мини-практика

Разделите существующие маршруты Internet Shop API на три отдельных Router.

Задание

- Создать каталог routes
- Создать productRoutes.js
- Создать categoryRoutes.js
- Создать orderRoutes.js
- Экспортировать каждый Router
- Импортировать routers в app.js
- Подключить их через app.use() с соответствующими префиксами

Критерии проверки

- В app.js больше нет маршрутов товаров, категорий и заказов
- Все существующие endpoints продолжают работать
- Маршруты разделены по ресурсам
- URL API после рефакторинга не изменились

Контрольные вопросы

1. Почему хранение всех маршрутов в одном `app.js` становится неудобным?
2. Для чего используется `express.Router()`?
3. Чем объект `router` отличается от основного объекта `app`?
4. Как зарегистрировать GET-маршрут внутри `Router`?
5. Зачем `Router` экспортируется из отдельного файла?
6. Для чего используется `app.use()` при подключении `Router`?
7. Как формируется полный URL из префикса `app.use()` и пути `router`?
8. Какой endpoint получится из `app.use("/api/products", router)` и `router.get("/:id")`?
9. Нужно ли запускать `app.listen()` внутри каждого файла `routes`?
10. Почему разделение маршрутов по ресурсам облегчает развитие приложения?

Итоги занятия

1

Express Router позволяет вынести группы связанных маршрутов в отдельные модули.

2

Router создаётся через `express.Router()`.

3

Маршруты внутри него регистрируются через `router.get()`, `router.post()` и другие методы.

4

Router экспортируется из файла и импортируется в основное приложение.

5

`app.use()` монтирует Router на определённый URL-префикс.

6

Полный endpoint состоит из префикса `app.use()` и локального пути Router.

7

Разделение маршрутов по ресурсам делает структуру Express-проекта понятнее и подготавливает приложение к дальнейшему разделению на `controllers` и `services`.